

CS6008 — Building a Secure Chat Application

Final technical report and verification plan

CS6008 — Building a Secure Chat Application

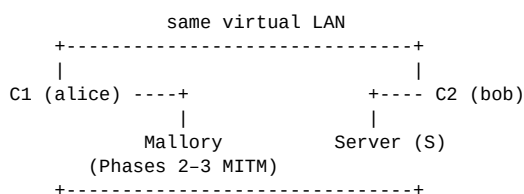
1. Introduction

The supplied assignment requires a one-to-one TCP chat application that is progressively hardened in five phases. Each phase adds one security property and is followed by an attack or verification demonstrating what the new mechanism does. The implementation is organized into phase1/ through phase5/, with a short README in each directory.

The protocol uses length-prefixed TCP frames: a 32-bit network-byte-order payload length followed by exactly that many bytes. This prevents message-boundary ambiguity on the TCP byte stream.

2. VM topology

Use four Ubuntu/Linux VMs for the full demonstration:



Record the actual IP addresses assigned to S, C1, C2 and Mallory in the final submitted report. The assignment explicitly requires separate VMs rather than loopback-only testing.

3. Phase 1 — Baseline chat

Implementation

The Phase 1 server accepts concurrent TCP connections, associates each connection with a username, maintains an online-user map, and relays messages. The client implements the required @username message, /chat username, /who, and /quit commands. The server does not encrypt or transform application messages.

Framing is a 4-byte length followed by payload. The server can theoretically support more than two clients because routing is performed by username lookup rather than a hard-coded C1/C2 pair.

Verification

The server is instrumented with [relay] sender -> recipient: message logging. Therefore, the server itself can read the complete message. A Wireshark capture on the VM network interface must show the same plaintext when TCP Stream is followed.

****Required evidence to insert:**** Phase-1 Wireshark TCP Stream screenshot showing the test message in plaintext.

4. Phase 2 — Diffie-Hellman and AES-GCM

DH implementation

The code uses the published RFC 3526 MODP Group 14 prime and generator 2. The exchange is implemented with low-level OpenSSL BIGNUM operations and BN_mod_exp; no OpenSSL DH object/API is used. Each client has an independent server-side exchange.

The raw DH value is SHA-256 hashed before becoming the 32-byte AES-256-GCM key. Hashing provides a fixed-size uniformly represented key-derivation output and avoids treating a raw group element encoding as a directly selected AES key.

Authenticated encryption

Every application frame after the exchange is protected by AES-256-GCM. A frame contains a fresh 96-bit random IV, ciphertext and 128-bit GCM authentication tag. Decryption rejects modified ciphertext when tag verification fails.

Fingerprint evidence

The implementation prints a truncated hash fingerprint rather than the raw shared secret or AES key. The local cryptographic test produced identical fingerprints at both DH endpoints:

```
f82f0435d4579fc9cdad38bdebb59fa6
```

The local tamper test reported:

```
tamper rejected: GCM authentication failed
```

MITM attack

The Phase-2 proxy performs two independent DH exchanges: one with the victim and one with the real server. It decrypts the victim-side ciphertext, prints the plaintext, then re-encrypts it under the independent server-side key. This demonstrates the central weakness: unauthenticated DH authenticates the mathematical exchange, not the identity of the peer.

A user would generally not notice because both legitimate endpoints complete apparently normal DH handshakes and continue to receive correctly decrypted messages. A trusted key fingerprint or authenticated server identity would be needed to expose the substitution.

****Required evidence to insert:**** Mallory VM log showing plaintext recovered, plus a Wireshark comparison between Phase 1 plaintext and Phase 2 ciphertext.

5. Phase 3 — PKI and proof of private-key possession

Certificate hierarchy

certs/make_certs.sh creates:

1. a 3072-bit CA private key and self-signed root certificate;
2. a 3072-bit server private key;
3. a server CSR with CN CS6008-Chat-Server;
4. a CA-signed server certificate;
5. a separate Mallory certificate used for the failed MITM demonstration.

The client receives the certificate before sending DH data. It validates the chain against its trusted CA, checks the certificate validity period through X.509 verification, and checks the expected common name. A failure aborts before DH.

Proof of possession

The server creates a fresh connection-specific nonce and signs it with the server private key. The client extracts the public key from the validated certificate and verifies the signature. Thus possession of only the certificate file is insufficient.

Verification

The legitimate client run produced:

- `[cert] CA signature, validity period and expected CN verified`
- `[proof] server private-key possession verified`
- matching client/server DH fingerprints can then be recorded.

The Mallory certificate substitution produced CERTIFICATE REJECTED, so the client stops before DH.

A second attack copies the legitimate server certificate but uses Mallory's private key to sign the proof. The client accepted the certificate chain and expected identity, then rejected the proof with PRIVATE-KEY POSSESSION REJECTED. This directly demonstrates the distinction between a certificate and possession of the matching private key.

Use the required command `openssl s_client -connect : -CAfile ../certs/ca.crt` as an additional certificate verification demonstration.

6. Phase 4 — End-to-end encryption

The client-to-client exchange is carried through the existing encrypted client-server channel but is not interpreted by the server. The required tags are used:

- `\_\_E2E\_INIT\_\_`
- `\_\_E2E\_ACK\_\_`
- `\_\_E2E\_MSG\_\_`

The relay continues to parse only the outer TO username payload routing envelope. After E2E establishment, the payload is an opaque tag and AES-GCM ciphertext. The server therefore knows who is talking to whom but cannot derive the C1-C2 key.

The local Phase-4 run independently computed the same E2E fingerprint on both clients:

36feb51eaf046ffbb0c93f05781d9733

A subsequent message was successfully displayed at the recipient as:

alice: secret-e2e-message

The server log contained an opaque `__E2E_MSG__`... value rather than the plaintext.

This is an additional inner encryption layer, not a replacement for the already encrypted client-server channel.

7. Phase 5 — Forward secrecy

The Phase-4 E2E session is extended with a 60-second automatic rekey timer. To prevent simultaneous rekey races, only the lexicographically smaller username initiates a rotation. The responder computes a fresh DH share and sends a rekey ACK. The initiator computes the same new key after the ACK. The active key is then replaced and the old key is no longer used for subsequent messages.

/rekey is included as a documented test command so two rotations can be demonstrated quickly without waiting two minutes.

The observed rotation evidence was:

The post-rotation messages after-rotation-1 and after-rotation-2 were decrypted correctly by Bob.

What forward secrecy adds

Phase 4 uses one long-lived E2E session key. If an attacker later obtains that key, they can decrypt recordings encrypted under that key. Phase 5 replaces the key with fresh DH-derived keys. A later compromise of the current key does not by itself reveal the earlier discarded keys, so previously recorded traffic from completed rotations is protected from that later single-key compromise. The attacker can still decrypt traffic protected by the compromised current key and can attack future sessions if they also gain the necessary ongoing secrets or endpoints; forward secrecy is specifically about limiting retrospective decryption after a later compromise.

8. Testing summary

Automated/local tests completed

- All five phase implementations compile with ``g++ -std=c++17 -O2 -Wall -Wextra`` and OpenSSL/libpthread linkage.
- Phase 2 DH endpoints produced the same fingerprint.
- AES-GCM tampering was rejected with an authentication failure.
- Phase 3 legitimate certificate validation and private-key proof succeeded.
- Phase 3 untrusted Mallory certificate was rejected.
- Phase 3 real-certificate/Mallory-private-key proof substitution was rejected.
- Phase 4 E2E fingerprints matched and encrypted chat was successfully delivered.
- Phase 5 showed two key rotations with matching fingerprints and successful post-rotation messages.

VM/Wireshark evidence still required from the deployment environment

The assignment explicitly requires separate VMs and Wireshark screenshots. Insert the following actual captures before submission:

1. Phase 1 plaintext TCP Stream.
2. Phase 2 ciphertext TCP Stream.
3. Phase 2 Mallory plaintext log.
4. Phase 3 valid certificate/proof flow.
5. Phase 3 Mallory certificate rejection.
6. Phase 3 copied-certificate/private-key mismatch rejection.
7. Phase 4 server log showing opaque E2E payload.
8. Phase 5 two rotations and post-rotation message delivery.

Do not fabricate these screenshots; capture them on the required VM topology.

9. Git checkpoints

The repository contains incremental commits for the five phases, including:

- Phase 1 baseline chat and PKI setup
- Phase 2 DH/AES-GCM and MITM demonstration
- Phase 3 certificate validation and proof of possession
- Phase 4 end-to-end client encryption
- Phase 5 forward secrecy key rotation

Run `git log --oneline` to show the checkpoint history.

10. AI-usage appendix

The assignment permits generative AI but requires an appendix listing prompts. The actual prompts used for this solution should be copied into the submitted appendix. Example categories are:

1. Request to implement the complete five-phase assignment from the supplied specification.
2. Request to organize C++ source code into independently executable phase directories.
3. Request to debug and test C++/OpenSSL compilation errors.
4. Request to explain the security rationale and verification evidence for DH, AES-GCM, PKI, E2E encryption and forward secrecy.
5. Request to prepare the final technical report and testing checklist.

11. Conclusion

The five phases demonstrate the progression from plaintext TCP messaging to authenticated server communication, client-to-client E2E confidentiality, and periodic forward-secret key rotation. The main security lesson is that confidentiality, authentication, end-to-end protection and forward secrecy solve different threats and must be evaluated against the specific attack each mechanism is intended to stop.